



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

**РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
НА ТЕМУ:
«Оптимизация эвристик решения уравнений в
словах
в SMT-решателе Ostrich»**

Студент _____
(Группа)

(Подпись, дата)

(И.О. Фамилия)

Руководитель НИР

(Подпись, дата)

(И.О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Уравнения в словах и методы их решения	5
1.1 Основные определения	5
1.2 Нильсеновские преобразования	5
1.3 Методы решения в современных SMT-решателях	6
1.4 Сложность задачи выполнимости уравнений в словах	7
2 SMT-решатели и задача определения однозначности шаблона	9
2.1 SMT-решатели и теория строк	9
2.2 Решатель Z3	9
2.3 Решатель Ostrich	13
2.4 Задача определения однозначности шаблона	13
2.5 Анализ существующих эвристик	14
2.6 Сравнительный анализ подходов к решению строковых ограничений	15
3 Оптимизированные эвристики на основе классов коммута- тивности	17
3.1 Проблема бесконечного ветвления	17
3.2 Вывод коммутативности переменных	18
3.3 Алгоритм нормализации уравнений	19
3.4 Встраивание эвристики в алгоритм Нильсена	20
4 Анализ подходов к встраиванию эвристики в SMT-решатель Ostrich	21
4.1 Архитектура модуля нормализации	21
4.2 Анализ корректности и завершаемости	22
ЗАКЛЮЧЕНИЕ	25
СПИСОК ЛИТЕРАТУРЫ	26

ВВЕДЕНИЕ

Задача проверки выполнимости уравнений в словах (word equations) занимает центральное место в теоретической информатике и находит широкое применение в статическом анализе программ, верификации протоколов безопасности и символьном исполнении кода. Уравнение в словах — это равенство двух слов, составленных из литералов и переменных, и задача состоит в нахождении такой подстановки строковых значений для переменных, при которой равенство выполняется.

В современной практике анализа программ уравнения в словах возникают при работе с SMT-решателями (Satisfiability Modulo Theories), поддерживающими теорию строк. Стандарт SMT-LIB 2.6 определяет набор операций над строками — конкатенацию, сравнение длин, взятие подстрок — которые формируют ограничения, сводящиеся к системам уравнений в словах. Такие решатели, как Z3 [1] и Ostrich [2], обрабатывают эти ограничения с использованием специализированных алгоритмов.

Одной из прикладных задач в области анализа строковых ограничений является задача определения однозначности шаблона (pattern ambiguity): по заданному регулярному выражению или шаблону требуется установить, существует ли строка, соответствующая шаблону более чем одним способом. Данная задача сводится к системе строковых ограничений и может быть передана SMT-решателю. Эффективность решения при этом критически зависит от качества эвристик, управляющих процессом разбиения уравнений.

Решатель Ostrich реализует эвристику нильсеновского разбиения (Nielsen splitting), основанную на классическом преобразовании Нильсена для уравнений в словах. Однако в ряде случаев данная эвристика порождает избыточные ветви доказательства, что приводит к экспоненциальному росту числа шагов вывода. Целью настоящей работы является разработка и реализация оптимизированной эвристики, использующей классы коммутативности нетерминалов для нормализации уравнений и сокращения пространства поиска.

Цель данной научно-исследовательской работы — изучение существующих эвристик решения уравнений в словах в SMT-решателях Z3 и Ostrich, анализ проблемы бесконечного ветвления при обработке уравнений с повторяющимися

переменными, разработка алгоритма нормализации на основе классов коммутации переменных и теоретическое обоснование его корректности.

1 Уравнения в словах и методы их решения

1.1 Основные определения

Пусть Σ — конечный алфавит, Σ^* — множество всех конечных слов над Σ , включая пустое слово ε . Множество Σ^* образует моноид относительно операции конкатенации. Пусть X — счётное множество переменных, $X \cap \Sigma = \emptyset$. Множество слов над расширенным алфавитом $\Sigma \cup X$ обозначается $(\Sigma \cup X)^*$.

Определение 1. Уравнение в словах над алфавитом Σ и множеством переменных X — это пара $(u, v) \in (\Sigma \cup X)^* \times (\Sigma \cup X)^*$, записываемая как $u = v$.

Подстановка $\sigma : X \rightarrow \Sigma^*$ продолжается до гомоморфизма $\hat{\sigma} : (\Sigma \cup X)^* \rightarrow \Sigma^*$ стандартным образом. Подстановка σ называется решением уравнения $u = v$, если $\hat{\sigma}(u) = \hat{\sigma}(v)$. Уравнение в словах выполнимо, если у него существует хотя бы одно решение.

Пример уравнения в словах: $x \cdot a \cdot y = b \cdot x \cdot z$, где $a, b \in \Sigma$ — литералы, $x, y, z \in X$ — переменные. Одним из решений является $x = b, y = b \cdot z$ при любом $z \in \Sigma^*$.

Задача разрешимости уравнений в словах была поставлена А. Марковым в 1954 году и оставалась открытой до работы Маканина [3], который доказал, что она разрешима. Впоследствии Пландовски [4] установил, что задача лежит в классе PSPACE. Задача разрешимости системы уравнений в словах является PSPACE-полной.

1.2 Нильсеновские преобразования

Метод Нильсена [5] для решения уравнений в словах основан на последовательном применении элементарных преобразований, уменьшающих длину уравнения. Рассмотрим уравнение $u = v$, где $u, v \in (\Sigma \cup X)^*$. Если оба слова непусты, то возможен один из следующих случаев для первых символов u и v :

1. первые символы совпадают — они сокращаются, и задача сводится к более короткому уравнению;
2. первый символ u — литерал a , первый символ v — переменная x — тогда x начинается с a , то есть $x = a \cdot x'$, и производится замена $x \mapsto a \cdot x'$;

3. первый символ u — переменная x , первый символ v — переменная y — тогда либо x является префиксом y ($x = x'$, $y = x \cdot y'$), либо y является префиксом x ($y = y'$, $x = y \cdot x'$).

Последний случай порождает ветвление при доказательстве: вводится новая переменная, и задача разбивается на два подслучая. Именно это ветвление является источником экспоненциального роста числа шагов в наихудшем случае.

В формальной записи нильсеновское разбиение при совпадении переменных x и y в начале слов u и v приводит к двум подзадачам:

$$x = y \cdot r \Rightarrow u[x \mapsto y \cdot r] = v[x \mapsto y \cdot r], \quad (1)$$

$$y = x \cdot r \Rightarrow u[y \mapsto x \cdot r] = v[y \mapsto x \cdot r], \quad (2)$$

где r — свежая переменная. Таким образом, метод строит дерево разбора, листьями которого являются либо противоречия, либо выполнимые тривиальные уравнения.

1.3 Методы решения в современных SMT-решателях

Современные SMT-решатели используют различные подходы к решению уравнений в словах. Можно выделить несколько основных парадигм [6, 7]:

1. Автоматные методы: уравнение кодируется конечным автоматом, и задача выполнимости сводится к непустоте языка пересечения автоматов. Данный подход реализован, в частности, в решателе Z3 [1].
2. Методы на основе преобразований: применяются нильсеновские или обобщённые преобразования, строящие дерево разбора. К этой категории относится решатель Ostrich.
3. Метод границ: для каждой переменной вводятся длинновые ограничения, и задача решается комбинацией теории строк и линейной арифметики.
4. DPLL(T) с теорией строк: расширение классического алгоритма DPLL для работы с теорией строк; используется в решателях CVC5 [8] и Z3 seq.

Каждый подход обладает своими преимуществами и ограничениями. Автоматные методы хорошо справляются с уравнениями, допускающими компактное автоматное представление, но плохо масштабируются при большом числе переменных. Методы на основе преобразований лучше работают с квадратными урав-

нениями (уравнениями, в которых каждая переменная встречается не более двух раз), характерными для задач однозначности шаблона.

1.4 Сложность задачи выполнимости уравнений в словах

Теоретическое изучение сложности задачи выполнимости уравнений в словах прошло несколько ключевых этапов, определивших современное понимание её трудоёмкости.

Первый фундаментальный результат принадлежит Г. С. Маканину [3], который в 1977 году доказал разрешимость задачи выполнимости одного уравнения в словах. Алгоритм Маканина основан на методе обобщённых уравнений (generalized equations): произвольное уравнение кодируется в специальный формат, в котором явно указаны позиционные перекрытия переменных. Доказывается, что если уравнение выполнимо, то решение может быть найдено за конечное число применений фиксированного набора разрешающих преобразований. Несмотря на то что исходный алгоритм Маканина имел неэлементарную сложность, сам факт разрешимости поставил задачу в разряд алгоритмически разрешимых.

В 2004 году В. Пландовски [4] существенно улучшил этот результат, доказав принадлежность задачи классу PSPACE.

Теорема 1 (Пландовски, 2004). *Задача выполнимости уравнения в словах над конечным алфавитом является PSPACE-полной.*

Аналогичный результат справедлив для систем уравнений в словах: добавление нескольких уравнений не меняет класса сложности — системы также PSPACE-полны.

Данные теоретические ограничения накладывают принципиальный предел на эффективность алгоритмов: полиномиального алгоритма для решения произвольных уравнений в словах (при предположении $P \neq \text{PSPACE}$) не существует. На практике, однако, задачи с небольшим числом переменных и структурированных ограничений решаются существенно быстрее. Это мотивирует разработку специализированных эвристик для конкретных классов уравнений.

Определение 2. *Уравнение в словах называется квадратным (quadratic), если каждая переменная встречается в объединении левой и правой частей ровно два*

раза. Уравнение называется линейным, если каждая переменная встречается не более одного раза.

Линейные уравнения решаются за линейное время методом последовательного применения подстановок. Квадратные уравнения формально PSPACE-трудны, однако их структура допускает применение специализированных алгоритмов. Для задачи определения однозначности шаблона с n переменными уравнение неоднозначности является квадратным: каждая из n переменных встречается в нём ровно два раза (по одному разу с каждой стороны равенства). Этот класс уравнений составляет основной предмет исследования данной работы.

2 SMT-решатели и задача определения однозначности шаблона

2.1 SMT-решатели и теория строк

SMT-решатели (Satisfiability Modulo Theories) — это системы автоматического доказательства теорем, позволяющие проверять выполнимость формул первого порядка относительно заданной теории. Стандарт SMT-LIB 2.6 [9] определяет единый интерфейс для работы с такими решателями, включая теорию строк `QF_S` с операциями конкатенации (`str.++`), длины (`str.len`), вхождения подстроки (`str.contains`), взятия подстроки (`str.substr`) и другими.

Формула теории строк в SMT-LIB 2.6 представляет собой булеву комбинацию строковых ограничений вида:

$$\varphi ::= (= t_1 t_2) \mid (\text{str.contains } t_1 t_2) \mid (\leq (\text{str.len } t) n) \mid \dots, \quad (3)$$

где t_1, t_2 — строковые термы, n — арифметическое выражение.

Задача выполнимости формулы φ в теории строк является PSPACE-полной, что накладывает теоретические ограничения на эффективность алгоритмов. На практике решатели используют эвристики, позволяющие быстро обрабатывать большинство встречающихся на практике задач.

2.2 Решатель Z3

Z3 [1] — универсальный SMT-решатель общего назначения, разработанный в Microsoft Research Леонардо де Моурой и Николасом Бйёрнером. Решатель поддерживает широкий спектр теорий: линейную и нелинейную арифметику над целыми и рациональными числами, теорию массивов, теорию битовых векторов, теорию неинтерпретированных функций, а также теорию строк. Z3 применяется в системах верификации программ (Dafny, Boogie), инструментах символьного исполнения, средствах проверки моделей и статического анализа программного обеспечения.

Архитектура DPLL(T). В основе Z3 лежит алгоритм DPLL(T) (Davis–Putnam–Logemann–Loveland modulo Theories) — расширение классического алгоритма DPLL для работы с теориями. Задача выполнимости формулы первого порядка разбивается на пропозициональную и теоретическую составляющие:

1. Пропозициональный решатель (SAT-ядро): атомарные ограничения заменяются булевыми переменными, формируя пропозициональный скелет. Ядро находит пропозиционально выполнимое присваивание или доказывает его отсутствие, используя механизм Conflict-Driven Clause Learning (CDCL).
2. Теоретический оракул (Т-решатель): при нахождении пропозиционально выполнимого присваивания набор соответствующих ограничений теории передаётся Т-решателю, который проверяет их консистентность. При обнаружении конфликта генерируется лемма конфликта — дизъюнкция отрицаний конфликтного подмножества ограничений.
3. Цикл взаимодействия: лемма конфликта добавляется в пропозициональную формулу, исключая данное присваивание. Процесс повторяется до нахождения выполнимого присваивания, согласованного с теорией, или доказательства невыполнимости.

Для комбинирования нескольких теорий Z3 применяет процедуру Нельсона–Оппена: Т-решатели попеременно обмениваются равенствами между общими переменными, добиваясь согласованной совместной модели.

Эволюция поддержки строк в Z3. Поддержка строковой теории в Z3 развивалась в нескольких версиях:

1. Z3-str [10] (2013): первый плагин для строк, реализованный как внешний Т-решатель. Взаимодействует с ядром Z3 через интерфейс теоретических плагинов. Применяет техники переписывания и упрощённые нильсеновские преобразования для уравнений конкатенации. Поддерживает конкатенацию, длину и базовые регулярные ограничения.
2. Z3-str2 [6] (2015): существенно улучшенная версия с оптимизированными эвристиками обработки строк. Введён более глубокий анализ длиновых ограничений, позволяющий на ранних этапах отсекал ветви с несовместными ограничениями на длины. Улучшена нормализация уравнений конкатенации.

3. Z3-str3 [11] (2017): переработанная версия с теоретически-осведомлёнными эвристиками (theory-aware heuristics). Введён режим «агрессивного длинового учёта»: ограничения на длины применяются не только для отсечки, но и для направленного вывода. Добавлены специализированные обработчики для операций подстроки, замены и вхождения.
4. Z3seq (с 2018 г.): текущая реализация в основной ветке Z3, основанная на теории последовательностей (theory of sequences). Строки рассматриваются как последовательности символов произвольного сорта, что позволяет унифицировать строковую теорию с более общим аппаратом последовательностей.

Теория последовательностей. В текущей версии Z3 строки обрабатываются в рамках теории последовательностей. Основные операции, доступные в формате SMT-LIB 2.6:

- `(str.++ t1 t2)` — конкатенация строк t_1 и t_2 ;
- `(str.len t)` — длина строки t ;
- `(str.substr t i n)` — подстрока длиной n с позиции i ;
- `(str.contains t1 t2)` — проверка вхождения t_2 в t_1 ;
- `(str.in_re t r)` — принадлежность строки t регулярному языку, задаваемому выражением r ;
- `(str.indexof t1 t2 i)` — позиция вхождения t_2 в t_1 начиная с позиции i ;
- `(str.replace t1 t2 t3)` — замена первого вхождения t_2 в t_1 на t_3 .

Для ограничений принадлежности регулярным языкам (`str.in_re`) Z3 строит конечный автомат по регулярному выражению и сводит задачу выполнимости к проверке непустоты его языка. Пересечение автоматов и проверка непустоты языка выполняются за время, пропорциональное произведению числа состояний.

Подход Z3 к уравнениям в словах. Для чистых уравнений в словах вида $u = v$, где $u, v \in (\Sigma \cup X)^*$, Z3 применяет комбинированный подход:

1. Упрощение: ограничения приводятся к нормальной форме правилами переписывания. Тривиальные равенства (оба операнда — константы) немедленно проверяются и разрешаются.

2. Длиновые ограничения: из каждого уравнения $u = v$ извлекается ограничение $\text{len}(u) = \text{len}(v)$, передаваемое арифметическому Т-решателю. Несовместность длиновых ограничений означает немедленную невыполнимость ветви.
3. Позиционный анализ: Z3 отслеживает позиционные соотношения: «строка x является префиксом y », « x и y начинаются с одного символа» и аналогичные. Это позволяет применять цепочки детерминированных правил вывода.
4. Ветвление в рамках DPLL(T): при невозможности применить детерминированные правила выполняется разбиение по случаям, аналогичное нильсеновскому ветвлению, но оформленное как добавление дизъюнктивных лемм в пропозициональный решатель.

Ограничения Z3 на квадратных уравнениях. Несмотря на универсальность, Z3 демонстрирует ряд ограничений при обработке квадратных уравнений с повторяющимися переменными:

- Автоматный подход, эффективный для регулярных ограничений, плохо масштабируется на уравнениях с большим числом переменных: пересечение автоматов требует числа состояний, экспоненциального от числа операндов.
- Длиновые эвристики Z3 не учитывают алгебраическую структуру уравнений — в частности, факт повторения переменной. Для уравнения $x \cdot x = y \cdot z$ Z3 не использует информацию о том, что длина x должна быть кратна сумме длин y и z , делённой на два.
- Для задач однозначности шаблонов с повторяющимися переменными Z3 в ряде случаев не завершает работу за разумное время, тогда как специализированные решатели, использующие нильсеновские преобразования, справляются с данным классом задач.

Таким образом, Z3 является мощным универсальным инструментом, ориентированным на широкий спектр задач, однако не специализированным для квадратных уравнений в словах. Это обосновывает актуальность специализированных решателей и оптимизированных эвристик нильсеновского разбиения, рассматриваемых в данной работе.

2.3 Решатель Ostrich

Ostrich [2] — SMT-решатель для строковых ограничений, разработанный в рамках фреймворка Princess [12]. Решатель реализован на языке Scala и использует процедуру Нельсона–Оппена для комбинирования теорий. Основной алгоритм решения строковых ограничений состоит из следующих компонентов:

1. Предварительная обработка: строковые ограничения, включая регулярные выражения и операции подстроки, преобразуются в систему уравнений-конкатенаций вида $\text{str} \cdot ++(x_1, x_2) = y$.
2. Нильсеновское разбиение: процедура NielsenSplitter применяет правила нильсеновского преобразования, строя дерево доказательства в фреймворке Princess.
3. Проверка длиновых ограничений: для каждого листа дерева доказательства проверяется совместность длиновых ограничений с помощью линейного программирования.
4. Проверка выполнимости: если длиновые ограничения совместны, строится конкретное решение.

Фреймворк Princess предоставляет механизм плагинов через интерфейс `Plugin.Action`. Основные типы действий:

- `Plugin.AxiomSplit` — создаёт разбиение: текущая цель делится на две подцели, каждая из которых содержит добавленную аксиому и список действий по удалению фактов;
- `Plugin.AddAxiom` — добавляет аксиому в текущую цель без создания дополнительных ветвей;
- `Plugin.RemoveFacts` — удаляет указанные факты из текущей цели.

2.4 Задача определения однозначности шаблона

Пусть Σ — алфавит, $\mathcal{P}$ — шаблон (pattern) над Σ , то есть слово над расширенным алфавитом $\Sigma \cup X$, где X — переменные. Говорят, что шаблон $\mathcal{P}$ неоднозначен (ambiguous), если существует строка $w \in \Sigma^*$, допускающая два различных соответствия шаблону:

$$\exists \sigma_1, \sigma_2 : X \rightarrow \Sigma^*, \quad \sigma_1 \neq \sigma_2, \quad \hat{\sigma}_1(\mathcal{P}) = \hat{\sigma}_2(\mathcal{P}) = w. \quad (4)$$

Задача определения однозначности сводится к проверке выполнимости уравнения:

$$\mathcal{P}[\sigma_1] = \mathcal{P}[\sigma_2] \wedge \sigma_1 \neq \sigma_2, \quad (5)$$

что при раскрытии ограничений $\sigma_1 \neq \sigma_2$ через длины или позиции переменных даёт систему строковых ограничений в теории строк.

Для шаблонов, содержащих повторяющиеся переменные, характерно наличие уравнений, где одна переменная встречается в обеих частях конкатенации. Например, для шаблона $x \cdot y \cdot x$ уравнение однозначности принимает вид:

$$x_1 \cdot y_1 \cdot x_1 = x_2 \cdot y_2 \cdot x_2, \quad (6)$$

что является квадратным уравнением в словах. Именно такие уравнения наиболее эффективно обрабатываются нильсеновскими методами.

2.5 Анализ существующих эвристик

Существующая реализация нильсеновского разбиения в Ostrich использует стандартное ветвление при встрече двух переменных в начале (или конце) обеих частей уравнения. При анализе задач однозначности шаблона наблюдается специфическая структура уравнений: одна переменная встречается дважды в одной части уравнения («удвоенная» переменная), тогда как в другой части стоят два различных символа.

Например, уравнение $x \cdot x \cdot r_1 = y \cdot z \cdot r_2$ имеет слева удвоенную переменную x , а справа — переменные y и z . В существующей реализации при стандартном нильсеновском разбиении для такого уравнения порождается ветвление, не использующее информацию об удвоении.

Ключевое наблюдение состоит в том, что при наличии двух нетерминалов x и z , встречающихся в обеих частях уравнения (или частично покрывающих друг друга), можно ввести понятие класса коммутативности и использовать его для нормализации уравнения, избегая тем самым ненужных ветвлений.

2.6 Сравнительный анализ подходов к решению строковых ограничений

Различные SMT-решатели реализуют принципиально разные стратегии обработки строковых ограничений. Рассмотрим ключевые характеристики трёх основных подходов.

Z3 (DPLL(T) + теория последовательностей). Архитектура Z3 обеспечивает хорошую обработку регулярных ограничений: для ограничений вида `str.in_re` строятся конечные автоматы, пересечение которых проверяется за полиномиальное время от размера автоматов. Система длиновых ограничений эффективно взаимодействует с арифметическим T-решателем. Слабое место — квадратные уравнения с повторяющимися переменными, для которых DPLL(T)-цикл не использует алгебраическую структуру уравнения.

CVC5 (DPLL(T) + теория строк). CVC5 [8] использует архитектуру DPLL(T) с встроенным T-решателем теории строк, разработанным на основе опыта Z3-str. Важная особенность — интеграция с квантором разрядности (bit-vector theory) позволяет эффективно обрабатывать ограничения на символьные коды. CVC5 поддерживает расширенный набор операций SMT-LIB 2.6, включая операции над строками в Unicode. Для уравнений с повторяющимися переменными CVC5 демонстрирует умеренную эффективность, аналогичную Z3.

Ostrich (Princess + нильсеновское разбиение). Ostrich [2] специализирован для уравнений конкатенации. Фреймворк Princess предоставляет механизм плагинов для теоретических процедур, а алгоритм нильсеновского разбиения непосредственно работает со структурой уравнений. Преимущество перед Z3 и CVC5 — возможность применять специализированные структурные эвристики, в том числе нормализацию через классы коммутации. Ограничение — потенциальное бесконечное ветвление при обработке уравнений с удвоенными переменными в базовой версии.

Сравнение по классам задач.

- Регулярные ограничения: Z3 и CVC5 эффективны за счёт автоматного аппарата; Ostrich поддерживает их, но без автоматной оптимизации.

- Чистые уравнения в словах: Ostrich наиболее эффективен благодаря нильсеновским преобразованиям; Z3 и CVC5 обрабатывают их в рамках общего DPLL(T)-цикла.
- Квадратные уравнения с повторяющимися переменными: базовый Ostrich может зависать; Z3 и CVC5 также испытывают трудности. Предложенная в данной работе нормализация через классы коммутации устраняет эту проблему для Ostrich.
- Задачи однозначности шаблонов: Ostrich с оптимизированными эвристиками является наиболее перспективным инструментом для данного класса, поскольку задача порождает квадратные уравнения, наиболее эффективно обрабатываемые нильсеновскими методами [13].

Таким образом, выбор Ostrich в качестве базового решателя для задачи однозначности шаблонов является обоснованным, а разработка оптимизированных эвристик нильсеновского разбиения составляет актуальную и практически значимую задачу.

3 Оптимизированные эвристики на основе классов коммутативности

3.1 Проблема бесконечного ветвления

При решении задачи однозначности шаблона уравнения принимают так называемую плоскую форму: уравнения-конкатенации содержат переменные, вхождения которых концентрируются в начале (или конце) обеих частей. Характерный пример — пара уравнений:

$$x_1 \cdot x_1 \cdot x_1 \cdot "A" \cdot y_1 \dots = Z, \quad x_2 \cdot x_2 \cdot x_2 \cdot "A" \dots = Z.$$

При $x_1 \neq x_2$ классический алгоритм Нильсена сталкивается с паттерном $a a \dots = b a \dots$, где $a = x_1$, $b = x_2$. Без специальной обработки алгоритм вынужден бесконечно ветвиться по префиксным перекрытиям переменных, что приводит к экспоненциальному росту дерева поиска или зависанию.

Рассмотрим подробнее, как возникает проблема. При первом применении нильсеновского разбиения к уравнению $x_1 \cdot x_1 \dots = x_2 \cdot x_2 \dots$ алгоритм рассматривает два случая:

- $x_1 = x_2 \cdot r_1$ (переменная x_2 является префиксом x_1);
- $x_2 = x_1 \cdot r_2$ (переменная x_1 является префиксом x_2).

В каждом из этих случаев в получившемся уравнении снова встречаются две переменные в начале обеих частей, и алгоритм вынужден снова ветвиться. При этом вновь введённые переменные r_1, r_2 наследуют ту же структуру, что и исходные, и дерево поиска продолжает расти. В результате возникает бесконечная цепочка ветвлений без гарантии завершения.

Ключевое наблюдение, лежащее в основе предлагаемой эвристики, состоит в следующем: если два нетерминала x_1 и x_2 образуют подобную бесконечно-ветвящуюся структуру, то они коммутируют, то есть $x_1 \cdot x_2 = x_2 \cdot x_1$. Это алгебраическое свойство позволяет переупорядочивать вхождения переменных в уравнении без изменения его выполнимости, что открывает возможность для нормализации и устранения избыточных ветвей.

3.2 Вывод коммутативности переменных

Покажем, что при многократном перекрытии переменные x_1 и x_2 неизбежно коммутируют. Рассмотрим уравнение:

$$x_1 \cdot x_1 \cdots = x_2 \cdot x_2 \cdots$$

Пусть $|x_1| > |x_2|$, тогда x_2 является префиксом x_1 . Положим $x_1 = x_2 \cdot x_{11}$. Подставляя в основное уравнение и сокращая первый x_2 , получаем новое уравнение вида:

$$x_{11} \cdot x_2 \cdot x_{11} \cdot x_1 \cdots = x_2 \cdots$$

Для него возникает новое ветвление. Пусть p — общий префикс x_{11} и x_2 . Возможны два подслучая:

- $|x_{11}| > |x_2|$: тогда $x_2 = p$;
- $|x_{11}| < |x_2|$: тогда $x_2 = p \cdot q \cdot x_{21}$ для некоторого x_{21} .

Продолжая ветвление для x_{21} , x_{31} , ... пока $|x_{11}| > |x_{2n}|$, получаем, что полная структура переменной x_1 имеет вид:

$$x_1 = x_{2,1} = (pq)^k \cdot p \cdot pq = (pq)^{k+1} \cdot p, \quad x_2 = (pq)^k \cdot p.$$

Выполним подстановку $x_1 = (pq)^{k+1}p$ и $x_2 = (pq)^k p$ обратно в уравнение $x_1 \cdot x_1 \cdots = x_2 \cdot x_2 \cdots$:

$$\text{Л.ч.: } (pq)^{k+1}p \cdot (pq)^{k+1}p \cdots = (pq)^k p \cdot qp \cdot (pq)^k p \cdot qp \cdots$$

$$\text{П.ч.: } (pq)^k p \cdot (pq)^k p \cdots = (pq)^k p \cdot pq \cdot (pq)^{k-1}p \cdots$$

Сокращая общий префикс $x_2 = (pq)^k p$, получаем равенство:

$$qp = pq.$$

Это доказывает, что x_1 и x_2 коммутируют: по лемме о коммутирующих словах [14] из $qp = pq$ следует, что p и q — степени одного примитивного слова t , то есть $x_1 = t^\alpha$ и $x_2 = t^\beta$ для некоторых $\alpha, \beta \geq 1$.

Определение 3. Множество переменных $\mathcal{C} = \{z_1, \dots, z_m\}$ образует класс коммутации, если для любых $z_i, z_j \in \mathcal{C}$ доказано $z_i \cdot z_j = z_j \cdot z_i$.

Таким образом, обнаружив в процессе нильсеновского разбиения условие $qr = pq$, алгоритм добавляет переменные в класс коммутации и далее использует это для нормализации уравнений.

3.3 Алгоритм нормализации уравнений

Для класса коммутации $\mathcal{C} = \{z_1, \dots, z_m\}$ и уравнения $U = V$ алгоритм нормализации работает следующим образом.

Листовые зоны — это максимальные под слова U (соответственно V), состоящие только из переменных класса $\mathcal{C}$. Обозначим $\nu_L(z)$ и $\nu_R(z)$ — число вхождений переменной $z \in \mathcal{C}$ в листовые зоны левой и правой части соответственно.

Шаги алгоритма нормализации:

1. Подсчитать $\nu_L(z)$ и $\nu_R(z)$ для каждого $z \in \mathcal{C}$.
2. Найти минимум: $m(z) = \min(\nu_L(z), \nu_R(z))$.
3. Отсортировать $\mathcal{C}$ по убыванию $m(z)$ и сформировать общий блок:

$$P = z_{\sigma(1)}^{m(z_{\sigma(1)})} \dots z_{\sigma(m)}^{m(z_{\sigma(m)})}.$$

4. Сформировать остатки: R_L содержит $\nu_L(z) - m(z)$ копий каждого z ; R_R — $\nu_R(z) - m(z)$ копий.
5. Сократить общий блок P из обеих частей. Нормализованное уравнение (после сокращения):

$$R_L \cdot U' = R_R \cdot V' \tag{7}$$

где U' и V' — нелистовые хвосты (в префиксном случае) или головы (в суффиксном случае) исходных частей.

Лемма 1. Нормализованное уравнение (7) эквивалентно исходному $U = V$ при условии коммутативности переменных класса $\mathcal{C}$: выполнимость сохраняется.

Пример 1. Рассмотрим уравнение $a^1 b^3 c^5 \dots = a^7 b^4 c^3 \dots$, где a, b, c коммутируют. Подсчитаем вхождения в листовых зонах:

Переменная	ν_L	ν_R	$m = \min$
a	1	7	1
b	3	4	3
c	5	3	3

Общий блок: $P = a^1 b^3 c^3$. Остатки: $R_L = c^2$, $R_R = a^6 b$. После сокращения P :

$$c^2 \dots = a^6 b \dots$$

Симметрия, порождавшая избыточные ветви, устранена.

3.4 Встраивание эвристики в алгоритм Нильсена

Эвристика нормализации встраивается в алгоритм нильсеновского разбиения в четыре шага:

1. Обнаружение коммутации. При доказательстве равенства $pq = qr$ переменные добавляются в класс коммутации $\mathcal{C}$.
2. Нормализация перед каждым расщеплением. Перед применением очередного нильсеновского шага вычисляется нормализация для всех текущих уравнений, содержащих переменные класса $\mathcal{C}$.
3. Немедленное отсечение. Если нормализованное уравнение противоречиво (например, R_L и R_R начинаются с разных литералов), ветвь немедленно отсекается без дальнейшего развёртывания дерева.
4. Откат состояния. При поиске с возвратами (backtracking) состояние классов коммутации корректно откатывается (roll-back) вместе с остальными фактами цели.

Практическая значимость: алгоритм избегает экспоненциального ветвления. Решатель способен обрабатывать более широкий класс шаблонов за конечное число итераций, тогда как базовая версия Ostrich уходила в бесконечный цикл на паттернах с повторяющимися переменными.

4 Анализ подходов к встраиванию эвристики в SMT-решатель Ostrich

4.1 Архитектура модуля нормализации

Предлагаемый алгоритм нормализации предназначен для встраивания в модуль `OstrichNielsenSplitter` в составе пакета `ostrich.proofops`. Данный модуль реализует интерфейс плагина `Princess` и подключается к основному циклу обработки целей доказательства (`handleGoal`).

В текущей архитектуре Ostrich обработка целей делится на три режима:

- Быстрые детерминированные правила — упрощение и декомпозиция по длинам; данный режим остаётся без изменений.
- Расщепление по алгоритму Нильсена — стандартный шаг нильсеновского разбиения, применяемый при отсутствии коммутарующих переменных.
- Нормализация с классами коммутации — предлагаемый новый режим; должен применяться приоритетно, если обнаружены уравнения с удвоенными нетерминалами.

Алгоритм обнаружения подходящих уравнений должен искать пары конкатенаций (l_1, res) и (l_2, res) с одинаковой правой частью `res`, у которых:

1. существует общая переменная x , входящая в начало (или конец) обеих левых частей l_1 и l_2 ;
2. в одной из левых частей переменная x встречается дважды подряд (удвоение).

При обнаружении таких уравнений строится ветвление с двумя ветвями. Ветвь коммутативности: обе исходных уравнения заменяются нормализованными формами $R_L \cdot U' = \text{res}$ и $R_R \cdot V' = \text{res}$ согласно алгоритму раздела 3.3. Ветвь разбиения: применяется стандартный нильсеновский шаг с введением свежей переменной.

Важно подчеркнуть порядок приоритетов режимов обработки. Предлагается следующая иерархия (от наивысшего к наименьшему приоритету):

1. Быстрые детерминированные правила (упрощение, сокращение одинаковых символов с концов уравнения, обнаружение тривиальных противоречий).
2. Нормализация с классами коммутации — новый режим, применяемый при обнаружении пары уравнений с удвоенными нетерминалами.
3. Стандартное нильсеновское разбиение — применяется в остальных случаях.

Данная иерархия гарантирует, что новый режим задействуется только тогда, когда стандартный алгоритм испытывает трудности, что минимизирует накладные расходы на уравнениях, не требующих нормализации.

Для реализации классов коммутации в рамках фреймворка Princess предлагается хранить класс коммутации C как множество переменных типа `ConstantTerm` (внутреннее представление переменных в Princess). Класс сохраняется как часть состояния плагина, передаваемого в каждый вызов `handleGoal`. При создании нового разбиения (`AxiomSplit`) классы коммутации копируются в обе ветви, что обеспечивает корректную изоляцию состояний.

Проверка условия вхождения переменной в класс коммутации сводится к поиску в множестве за время $O(|C|)$. Подсчёт $\nu_L(z)$ и $\nu_R(z)$ выполняется однократным проходом по листовой зоне уравнения за время $O(|U| + |V|)$, что не влечёт существенных накладных расходов на практике.

4.2 Анализ корректности и завершаемости

Для оценки предложенного алгоритма рассмотрим три класса задач в формате SMT-LIB 2.6:

1. Простые уравнения с удвоенными переменными: $x \cdot x = y \cdot z$, $x \cdot x \cdot r = y \cdot z \cdot r$.
2. Задачи определения однозначности шаблонов: для шаблона с повторяющимися переменными генерируется уравнение однозначности.
3. Задачи без удвоенных переменных: контрольный класс для проверки отсутствия регрессий.

Для класса задач без удвоенных переменных алгоритм нормализации не применяется: условие обнаружения не выполняется, и решатель работает по стан-

дартному нильсеновскому пути. Это гарантирует, что введение нового режима не нарушает поведение решателя на уже корректно решаемых задачах.

Для класса задач с удвоенными переменными введение классов коммутации обеспечивает строгую завершаемость (termination): процесс бесконечного порождения ветвей дерева Нильсена сокращается до конечного числа итераций. Это следует из того, что нормализация канонически упорядочивает переменные класса коммутации, устраняя симметрию, которая в базовой версии порождала бесконечно расходящиеся ветви поиска.

Корректность нормализации. Лемма о корректности нормализации утверждает, что переход от уравнения $U = V$ к нормализованному уравнению $R_L \cdot U' = R_R \cdot V'$ сохраняет выполнимость. Доказательство основано на следующих соображениях:

1. Переменные класса коммутации $\mathcal{C}$ коммутируют попарно, поэтому их произвольная перестановка не меняет результата конкатенации (при условии коммутативности).
2. Общий блок P выбирается как наибольший блок, присутствующий одновременно в листовых зонах обеих частей. Его сокращение из обеих частей уравнения эквивалентно применению обратимой подстановки.
3. Остатки R_L и R_R однозначно определяются по числам вхождений $\nu_L(z)$, $\nu_R(z)$ и минимуму $m(z)$, что делает нормализацию детерминированной.

Таким образом, нормализованное уравнение выполнимо тогда и только тогда, когда выполнимо исходное (при фиксированном классе коммутации).

Завершаемость расширенного алгоритма. Алгоритм нильсеновского разбиения с интегрированной нормализацией завершается за конечное число шагов на задачах с удвоенными переменными. Ключевой аргумент: нормализация строго уменьшает суммарное число вхождений переменных класса коммутации в уравнении (сокращается общий блок P), а при выполнении условия немедленного отсеечения (когда R_L и R_R начинаются с разных литералов) ветвь закрывается без дальнейшего развёртывания. Поскольку число вхождений переменных ограничено (уравнение конечно), нормализация может применяться не более конечного числа раз до достижения тривиального уравнения или противоречия.

Корректность отката состояния. Механизм отката (backtracking) в фреймворке Princess основан на иммутабельных структурах данных: состояние цели (goal) в каждой ветви доказательства является снимком, независимым от других ветвей.

Классы коммутации хранятся как часть состояния цели и автоматически откатываются при возврате к родительской ветви. Это обеспечивает корректность работы алгоритма в режиме поиска с возвратами без дополнительных механизмов управления состоянием.

ЗАКЛЮЧЕНИЕ

В данной научно-исследовательской работе изучена задача решения уравнений в словах, её постановка и методы решения, применяемые в современных SMT-решателях. Рассмотрены автоматные методы, нильсеновские преобразования и $DPLL(T)$ -подходы; проведён анализ алгоритма решателя Ostrich, основанного на фреймворке Princess.

Выявлено, что стандартное нильсеновское разбиение при обработке уравнений с удвоенными нетерминалами, характерных для задач определения однозначности шаблона, порождает бесконечное ветвление. Показано, что многократное перекрытие двух переменных в процессе нильсеновского разбиения неизбежно приводит к условию $qr = pq$, что доказывает их коммутативность: обе переменные являются степенями одного примитивного слова.

На основе этого теоретического результата разработан алгоритм нормализации уравнений с помощью классов коммутации: вычисляется общий блок P по минимальным числам вхождений переменных в листовые зоны обеих частей уравнения, общий блок сокращается, и вместо исходного уравнения с произвольным порядком коммутирующих переменных получается нормализованная форма $R_L \cdot U' = R_R \cdot V'$. Доказана корректность нормализации: выполнимость исходного уравнения сохраняется. Описана схема встраивания алгоритма в цикл нильсеновского разбиения с корректным откатом состояния при поиске с возвратами.

Дальнейшими направлениями развития работы являются: реализация алгоритма в модуле `OstrichNielsenSplitter` решателя Ostrich, расширение понятия класса коммутации на произвольное число переменных, интеграция с длиновыми ограничениями для ранней отсечки ветвей, а также применение нормализации к задачам проверки регулярных ограничений.

СПИСОК ЛИТЕРАТУРЫ

1. Moura L. de, Bjørner N. Z3: An efficient SMT solver // Tools and Algorithms for the Construction and Analysis of Systems (TACAS). — Springer, 2008. — С. 337—340.
2. Decision Procedures for Path Feasibility of String-Manipulating Programs with Complex Operations / T. Chen [и др.] // Proceedings of the ACM on Programming Languages (POPL). Т. 6. — 2022. — С. 1—30.
3. Makanin G. S. The problem of solvability of equations in a free semigroup // Mathematics of the USSR-Sbornik. — 1977. — Т. 32, № 2. — С. 129—198.
4. Plandowski W. Satisfiability of word equations with constants is in PSPACE // Journal of the ACM. — 2004. — Т. 51, № 3. — С. 483—496.
5. Nielsen J. Die Isomorphismengruppe der freien Gruppen // Mathematische Annalen. Т. 91. — 1924. — С. 169—209.
6. Zheng Y., Zhang X., Ganesh V. Z3-str 2: An improved string constraint solver // Tools and Algorithms for the Construction and Analysis of Systems (TACAS). — Springer, 2015. — С. 402—416.
7. An efficient string solver with loop-free abstractions / T. Liang [и др.] // Tools and Algorithms for the Construction and Analysis of Systems (TACAS). — Springer, 2016. — С. 178—196.
8. cvc5: A Versatile and Industrial-Strength SMT Solver / H. Barbosa [и др.] // Tools and Algorithms for the Construction and Analysis of Systems (TACAS). — Springer, 2022. — С. 415—442.
9. Barrett C., Fontaine P., Tinelli C. The SMT-LIB Standard: Version 2.6 // Technical Report, The University of Iowa. — 2017.
10. Zheng Y., Zhang X., Ganesh V. Z3-str: A Z3-Based String Constraint Solver // Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE). — ACM, 2013. — С. 539—542.

11. Berzish M., Ganesh V., Zheng Y. Z3str3: A String Solver with Theory-Aware Heuristics // Formal Methods in Computer-Aided Design (FMCAD). — IEEE, 2017. — С. 55—59.
12. Rümmer P. A Constraint Sequent Calculus for First-Order Logic with Linear Integer Arithmetic // Logic for Programming, Artificial Intelligence, and Reasoning (LPAR). — Springer, 2008. — С. 274—289.
13. String Constraints for Verification / P. A. Abdulla [и др.] // Computer Aided Verification (CAV). — Springer, 2014. — С. 150—166.
14. Хмелевский Ю. И. Об уравнениях в свободных полугруппах и группах // Труды Математического института имени В.А. Стеклова. — 1971. — Т. 107. — С. 1—324.